

# Text is evidence: keep it before conversion

Daily learning note | 5 October 2026 | C003a intuition

Sidiq review status: Not yet reviewed.

## Research question and hypothesis

Can a whole-year savings function discover that its integer argument originally came from a fractional value? Hypothesis: once 2.5 has been converted to the integer 2, the function has lost the evidence needed to reject the original request. Validation must examine the original input.

C002 already calculates annual compound growth and rejects negative year counts. Its September 30 note identified conversion before the function call as a limitation. Today's curriculum slot studies that one prerequisite. The bounded improvement is a compiled, reproducible teaching counterexample with a hand classification worksheet. The years parser remains a later step in this week's slice.

## Beginner explanation and vocabulary

Imagine a form with a box labelled whole years. The written marks are text. Before doing arithmetic, we must decide whether all those marks describe an allowed count. Throwing away a decimal part or ignoring an unexplained letter changes the request rather than checking it.

**Text:** a sequence of characters, such as quoted "2.5". **Value:** a number used in arithmetic. **int:** C++'s signed integer type; it has a finite range and stores no fractional part. **double:** a floating-point type that can represent fractions, with limited precision and range. **Parsing:** interpreting text as a value.

**Validation:** checking the interpretation against the model's rules. **Conversion:** changing a value's type. These are different jobs.

**Principal P:** opening money in arbitrary currency units. **Annual rate r:** effective interest per model year, written as a fraction; 0.05 means 5%. **Years n:** a nonnegative whole count. **Balance B:** closing money. These are model years, not dates or a market calendar.

## Classify the original input by hand

Quotes below mark text boundaries and are not part of the supplied input. The decisions describe the intended whole-year model; no parser has yet produced these outcomes.

| Original text | Decision        | Explanation                                      |
|---------------|-----------------|--------------------------------------------------|
| "2"           | Accept: 2 years | A nonnegative whole count                        |
| "2.5"         | Reject          | A fractional count lies outside this model       |
| "-1"          | Reject          | Negative years lie outside this model            |
| "" (blank)    | Reject          | Missing input does not mean zero years           |
| "2x"          | Reject          | The unexplained suffix prevents a complete count |

Whitespace, signs, leading zeros and the precise integer limit are tomorrow's grammar decisions. Do not invent their policy from today's five examples. Zero itself is a legitimate count in C002; no elapsed year leaves the principal unchanged. Blank input supplies no such count.

## Mathematics and a worked example

For whole  $n$  under fixed annual compounding,  $B = P(1+r)^n$ . Start with  $P=1000$  and  $r=0.05$ . After one year, interest is  $1000 \times 0.05 = 50$  and balance is 1050. After the second year, interest is  $1050 \times 0.05 = 52.50$  and balance is 1102.50. The two-year interest total is 102.50 units.

Now start with the numeric value 2.5. Converting it to int produces 2 in this example. C002 receives 2 and returns 1102.50. That answer is mathematically correct for two years. It is not a validated answer to the original fractional-year request. A whole-year contract does not choose a partial-year interest convention; do not silently introduce one by extending the exponent or discarding a fraction.

## Introduce the C++ syntax before using it

`std::string` is the standard text type, supplied by the string header. A quoted literal supplies its characters. `const` prevents reassignment of the named variable. A semicolon ends a statement. `static_cast<int>(value)` explicitly requests conversion of a numeric value to int. The cast does not check our whole-year policy. Only the safe, fixed value 2.5 is converted in this demonstration.

Read the three declarations in `text_vs_years.cpp` in order:

1. `const std::string original_text = "2.5";` retains the original characters.
2. `const double fractional_years = 2.5;` separately creates the numeric value. This line does not parse the string above it; the two literals are deliberately written independently.
3. `const int converted_years = static_cast<int>(fractional_years);` discards the fraction, leaving 2.

The existing int `main()` entry point and braces enclose the demonstration. The includes provide the savings declaration, formatting, output and text facilities. The first three `std::cout` statements print the text, separate number and converted integer. Each `<<` sends another piece to the stream; `\n` ends the output line. The formatting statement uses two decimal places, as in the existing demo.

The next statement calls `compound_balance(1000.0, 0.05, converted_years)` and prints its balance. The last line prints the explanation of what was lost. Reaching the end of `main` reports success. The project worksheet walks through the same short source; no branches or parser APIs are introduced.

## Assumptions, costs and failure cases

Assume fixed effective annual interest, no external deposits or withdrawals and no fees, tax or inflation. Costs would reduce wealth; the model does not estimate them. There is no observed market return, trade, statistical sample, parameter search or profitability claim. These invented numbers illustrate a type boundary. Currency amounts printed to two decimals do not establish exact-money arithmetic.

A correct function can still answer the wrong question when its caller has changed the input. For this interface, a negative count is rejected but a lost fraction cannot be recovered. Blank and trailing-text cases remain hand expectations until a parser implements them. Arbitrary floating-to-int conversion is unsafe as a range check; out-of-range conversion can have undefined behaviour.

## Data and evaluation protocol

All inputs are fixed synthetic teaching values, with no market dates or vendor data. Before running, set the expected five output lines by hand: original text 2.5, separate numeric value 2.5, converted integer 2, balance 1102.50 and an explanation that the function cannot recover the fraction. Compare complete executable output twice against that stored expectation. Independently update a Python Decimal balance with two annual interest credits. Retain the existing full project checks.

This evaluates reproducibility, the conversion observation and one arithmetic example. It does not measure how many invalid text forms a parser rejects, because none is implemented. The hand table has one intended acceptance and four intended rejections; they are not five automated parser tests.

## Implementation, results and reproduction

Added `cpp/projects/savings_growth/text_vs_years.cpp`, `text_vs_years_expected.txt` and `text_vs_years.md`. The existing savings implementation is reused unchanged. Make adds `cpp-text-years` and includes the new exact output comparison in `cpp-check`, so the normal `make check` command exercises it too.

| Check                       | Observed result                                                       |
|-----------------------------|-----------------------------------------------------------------------|
| Fixed conversion            | 2.5 becomes integer 2                                                 |
| C002 call after conversion  | 1102.50 units for two years                                           |
| Independent Decimal credits | 1050 then 1102.50, agreeing with displayed balance                    |
| Two executable replays      | Both match the hand-authored output exactly                           |
| Existing C++ function suite | 66 checks pass; existing demonstration still matches                  |
| Full Python suite           | 608 tests pass                                                        |
| Code quality                | Ruff lint, 49 formatted files and strict mypy on 21 source files pass |
| Build and dependencies      | Fresh warning-clean C++17 build and pip consistency pass              |

Run `make cpp-text-years` for the new example, `make check` for all checks, and `make -B cpp-check` for a fresh C++ build. The build treats Wall, Wextra and Wpedantic warnings as errors. The explicit cast keeps the deliberate conversion visible; compiler acceptance is not domain validation. No new dependency, general coverage score or personal mastery is claimed. The implementation checks passed on their first run. Rebuild the report with `make note NOTE_DATE=2026-10-05` and inspect its PDF pages.

## Primary sources consulted on 5 October

Microsoft's standard-conversions documentation supports fractional truncation toward zero and the representability restriction. We use its conversion rule; the whole-year model policy is our choice. <https://learn.microsoft.com/en-us/cpp/cpp/standard-conversions?view=msvc-170>

Microsoft's string/character-literals documentation explains quoted string literals. Our stored text and separately written numeric literal make the distinction visible without implementing parsing. <https://learn.microsoft.com/en-us/cpp/cpp/string-and-character-literals-cpp?view=msvc-170>

## Interview explanation and limitations

A concise explanation: the function sees only the integer it receives. If a caller first converts 2.5 to 2, a nonnegative check cannot distinguish that value from an honestly supplied 2. Preserve and validate the original text before constructing the integer. Then apply the financial model's rules.

The teaching executable intentionally has no external-input interface. It proves the observation for one safe fixed value, not correctness of a future parser. The worksheet introduces full-input meaning but leaves detailed grammar and range choices for tomorrow. C003a remains in progress; C003b full CLI, vectors and statistics remain later prerequisites. Completion of a note or build does not demonstrate Sidiq's independent understanding; personal review is still Not yet reviewed.

## Next step, cadence and necessary input

October 5's intuition slice is complete. October 6 defines the years-only grammar, full consumption, whitespace and range; October 7 implements one parser function. Stay with one concept per day and repeat prerequisites if needed. The October 12 ledger still depends on function/branch readiness.

Today's shared London-date draw is 1. Actual programme history showed zero commits before this unit, leaving one slot. Publish this coherent lesson with its evidence as one commit after final PDF and staged-diff checks; report the actual hash, remote verification and count in the briefing and automation memory. Publication authorisation is separate from personal review.

No input is required for tomorrow's synthetic lesson. Q004a lawful entitlement/sample evidence and Q005b integration gates still apply before empirical work. No data acquisition or narrowed claim has been approved. The daily schedule remains 08:00 Europe/London with daylight-saving changes. Stop new research after 31 December 2026 and request the next programme scope.

## Check your understanding

1. Why are quoted "2.5" and unquoted 2.5 different in the C++ example?
2. Why can C002 reject -1 years but fail to detect that 2 came from 2.5?
3. Which of "2", "2.5", "-1", blank and "2x" should be accepted by the intended model, and why?
4. Work through two years from 1000 at 5%. Does the resulting balance answer a 2.5-year request?

## Suggested answers

1. The quoted form supplies characters stored as text. The unquoted form supplies a double numeric value. Today's example writes them separately; it performs no text parsing.
2. The negative integer is still visible to the function. The fraction has already been discarded before the call, so the received 2 contains no evidence of its earlier value.
3. Only "2" is accepted in this set. The others are fractional, negative, missing or contain an unexplained suffix. These are hand decisions, not automated parser results. Blank is not zero.
4. First interest is 50, giving 1050. Second interest is 52.50, giving 1102.50. That answers two whole years only. A fractional-year request is outside the stated contract and must not silently become two.